

# Blueprint — ELI Eval Harness

## Contents

|                                                                             |   |
|-----------------------------------------------------------------------------|---|
| 1. Directory tree . . . . .                                                 | 1 |
| 2. The contract everything depends on: the normalised result dict . . . . . | 1 |
| 3. File-by-file contracts . . . . .                                         | 2 |
| 4. Data flow . . . . .                                                      | 3 |
| 5. How to extend . . . . .                                                  | 4 |
| 6. Run commands . . . . .                                                   | 4 |
| 7. Design invariants (don't break these) . . . . .                          | 4 |

Exact structural guide to tools/eval/. The harness drives ELI's **real pipeline** and asserts on its **own telemetry** (action, matched\_by, grounding\_confidence, response\_mode, latency) plus answer text. 100% local, offline by default, no cloud judge, no required extra deps (PyYAML only, already a project dep).

Two tools share one driver: **run\_eval.py** (behavioural PASS/FAIL board) and **profile\_runtime.py** (runtime-graph profiler — replays the stored conversations + cases under a sys.settrace line tracer to report exact line-% hit, action coverage, which of the 14 agents fire, and a hot/cold module heatmap with a ranked cold-file cut-list). Full run commands for both are in tools/eval/README.md.

---

## 1. Directory tree

```
tools/
├── __init__.py          # makes `tools` importable
└── eval/
    ├── __init__.py
    ├── eli_driver.py    # THE shared core – only file that calls ELI
    ├── assertions.py    # deterministic checks over a driver result
    ├── cases.yaml       # the test cases (data, not code)
    ├── run_eval.py      # pure-Python green/red board (canonical)
    ├── profile_runtime.py # runtime-graph profiler (what executes / dead code)
    ├── README.md        # quickstart
    └── promptfoo/
        ├── eli_provider.py # optional Node UI front-end
        └── promptfooconfig.yaml # thin adapter → eli_driver
```

Two front-ends (run\_eval.py, promptfoo/) sit over **one** driver (eli\_driver.py). Neither front-end knows how to call ELI; only the driver does. Add a third front-end by importing the driver — never by re-implementing the call.

---

## 2. The contract everything depends on: the normalised result dict

Both driver entry points return a plain dict with this shape. Assertions and the promptfoo provider read **only** these keys, so the rest of the harness is insulated from ELI's internal return shapes.

| key                     | type                | router     | engine | meaning                                                            |
|-------------------------|---------------------|------------|--------|--------------------------------------------------------------------|
| target                  | str                 | -          | -      | "router" or "engine"                                               |
| action                  | str                 | -          | -      | routed/executed action (e.g. NEXT_MEDIA)                           |
| matched_by              | str                 | -          | -      | router rule id (e.g. media.app_targeted)                           |
| confidence              | float               | -          | -      | route / response confidence                                        |
| args                    | dict                | -          | —      | router action args                                                 |
| text                    | str                 | ""         | -      | the answer shown to the user                                       |
| grounding_response_mode | float   None<br>str | None<br>"" | -<br>- | trace.grounding_conf<br>meta.response_mode (e.g. ungrounded_hedge) |
| latency_s               | float               | -          | -      | wall-clock for the call                                            |
| skipped                 | bool (engine)       | —          | -      | True when no model → case SKIPS, not fails                         |
| raw                     | original            | -          | -      | the untouched router dict / process() result                       |

### 3. File-by-file contracts

#### eli\_driver.py — the shared core

The only module that imports ELI internals.

- `route_only(prompt: str, network: Optional[bool]=None) -> dict` Calls `eli.execution.router_enhance`. **No model required.** Fast (ms). Returns the normalised dict with `target="router"`.
- `run_engine(prompt, network=None, reasoning_mode="quick", session_id=None) -> dict` Runs the full `CognitiveEngine.process(...)`. **Needs a model loaded.** Normalises str / dict / generator returns. On engine-init failure returns `{"skipped": True, "reason": "no engine/model"}` so cases SKIP, not crash.
- `get_engine()` — lazily builds & caches one headless `CognitiveEngine(auto_init_gguf=True)`; returns None if it can't init. Model comes from `config/settings.json` (swap there to compare models).
- `_network(state)` — context manager that monkeypatches `eli.core.config.network_allowed` for the call **only**, then restores it. **Non-persisting** — running the eval never flips the user's real offline setting. `state=None` leaves it as-is.

## assertions.py — deterministic checks

- `check(assertion: dict, result: dict) -> (passed: bool, detail: str)` No LLM judge. Reads only the normalised result dict.

| type                                       | value                   | passes when                             |
|--------------------------------------------|-------------------------|-----------------------------------------|
| <code>contains / not_contains</code>       | <code>str</code>        | (case-insensitive) substring of text    |
| <code>regex</code>                         | <code>pattern</code>    | <code>re.search</code> on text          |
| <code>action_is / action_not</code>        | <code>str   list</code> | <code>action ∈ / ∉ list</code>          |
| <code>matched_by</code>                    | <code>str</code>        | substring of <code>matched_by</code>    |
| <code>response_mode</code>                 | <code>str   list</code> | <code>response_mode ∈ list</code>       |
| <code>grounding_min / grounding_max</code> | <code>float</code>      | <code>grounding ≥ / ≤ value</code>      |
| <code>hedged</code>                        | <code>bool</code>       | answer is an honest “won’t guess” hedge |
| <code>max_latency_s</code>                 | <code>float</code>      | <code>latency_s ≤ value</code>          |

## cases.yaml — the test data

List of case dicts:

```
- id: next_track_real          # unique
  target: router               # router (fast, model-free) | engine (needs model)
  prompt: "next track"
  network: on                  # optional: on|off – forced for this case only
  mode: quick                  # optional engine reasoning mode (default quick)
  assert:
    - {type: action_is, value: [NEXT_MEDIA, MEDIA_CONTROL]}
```

## run\_eval.py — the board

CLI over the driver. Flags: `--target router|engine|all` (default router) `--cases <path>` (default `cases.yaml` next to the script) `--filter <substr>` (match on case id) `--json <path>` (machine-readable records)

Prints PASS/FAIL/SKIP per case + failing-assertion detail + a summary line. **Exit code is non-zero if any case fails** → drop into CI / pre-push.

## promptfoo/eli\_provider.py — optional Node front-end

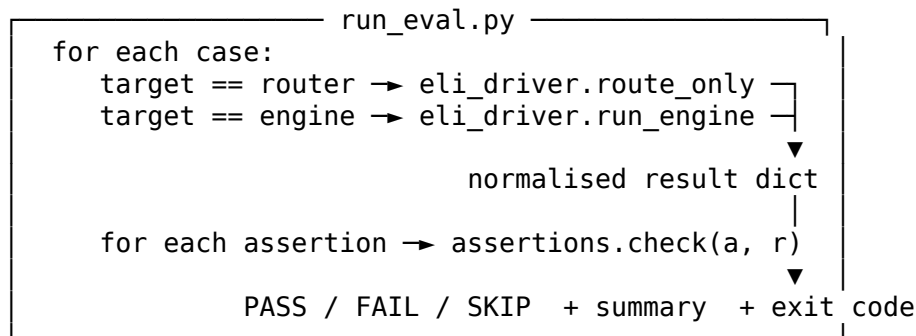
`call_api(prompt, options, context) -> {"output": text, "metadata": {action, matched_by, grounding, response_mode, latency_s, skipped}}`. Thin adapter that calls `eli_driver.run_engine`. Network/mode via test vars.

## promptfoo/promptfooconfig.yaml

Declares the provider + sample tests. Deterministic text asserts work out of the box; routing/grounding asserts read metadata. **Keep any llm-rubric judge pointed at a LOCAL model — never a cloud API.**

## 4. Data flow

```
cases.yaml
| (id, target, prompt, network, assert[])
▼
```



```
promptfoo eval → eli_provider.call_api → eli_driver.run_engine → same dict
```

route\_only and run\_engine are the only doors into ELI. Everything upstream (cases, assertions, boards, promptfoo) is plumbing over the normalised dict.

## 5. How to extend

**Add a case** — append to `cases.yaml`. Rule of thumb: *every bug a log reveals becomes a case*. Pick target: router if it's a routing decision (fast, no model); target: engine if it needs an answer / grounding / latency.

**Add an assertion type** — add a branch in `assertions.py::check` returning `(bool, detail)`. It automatically reads the normalised dict, so no driver change is needed.

**Surface a new ELI signal** — add the key in `eli_driver` (both `route_only` and/or `run_engine` return dicts), then assert on it. Example: to assert on `agents_used`, read it from `raw` in the driver and expose it as a top-level key.

**Compare two models** — set model A in config/settings.json, run `--target all --json a.json`; swap to model B, `--json b.json`; diff. Watch grounding, hedged, correctness, latency\_s. This is how “is the 24B worth the CPU latency on this box” becomes a number.

## 6. Run commands

```
cd ~/Desktop/ELI MKXI-main MAY NEWEST && source .venv/bin/activate
```

```
python tools/eval/run_eval.py           # router board, ~60ms, no model
python tools/eval/run_eval.py --target all # + engine cases (needs a model)
python tools/eval/run_eval.py --filter media # subset by id
python tools/eval/run_eval.py --target all --json out.json
python tools/eval/run_eval.py || echo "EVAL FAILED" # pre-push guard
```

```
# optional Node UI:
```

```
cd tools/eval/promptfoo && npx promptfoo@latest eval && npx promptfoo@latest view
```

Sanity: the bare `run_eval.py` should print 10 passed 0 failed 0 skipped. Import error  $\Rightarrow$  not at repo root or venv not active.

## 7. Design invariants (don't break these)

1. **One driver.** Only `eli_driver.py` imports ELI internals. Front-ends go through it.
2. **Normalised dict is the contract.** Assertions/providers never touch ELI's raw return shapes except via `raw`.

3. **Network state is per-call and non-persisting.** Never write `network_enabled` to settings from the harness.
4. **No cloud judge.** Deterministic assertions first; any LLM-graded check uses a local model. Offline by default.
5. **Engine cases degrade to SKIP**, never crash, when no model is present — so the router board always runs in CI.
6. **Cases are data.** New coverage = new YAML, not new code (unless it's a new assertion type or signal). ""